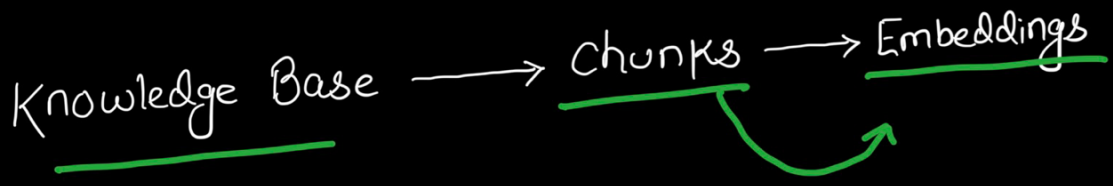


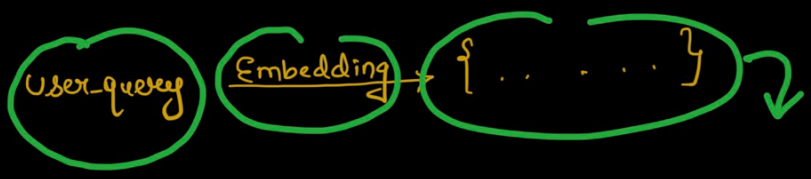
codestorywithmiK

CswithMIK

codestorywithMIK

Video-15 Recap





chunks = [{ ___ }, { ___ }, { ___ } . . ~]

We can now compare user_query embedding with chunks embeddings and find relevant informations.

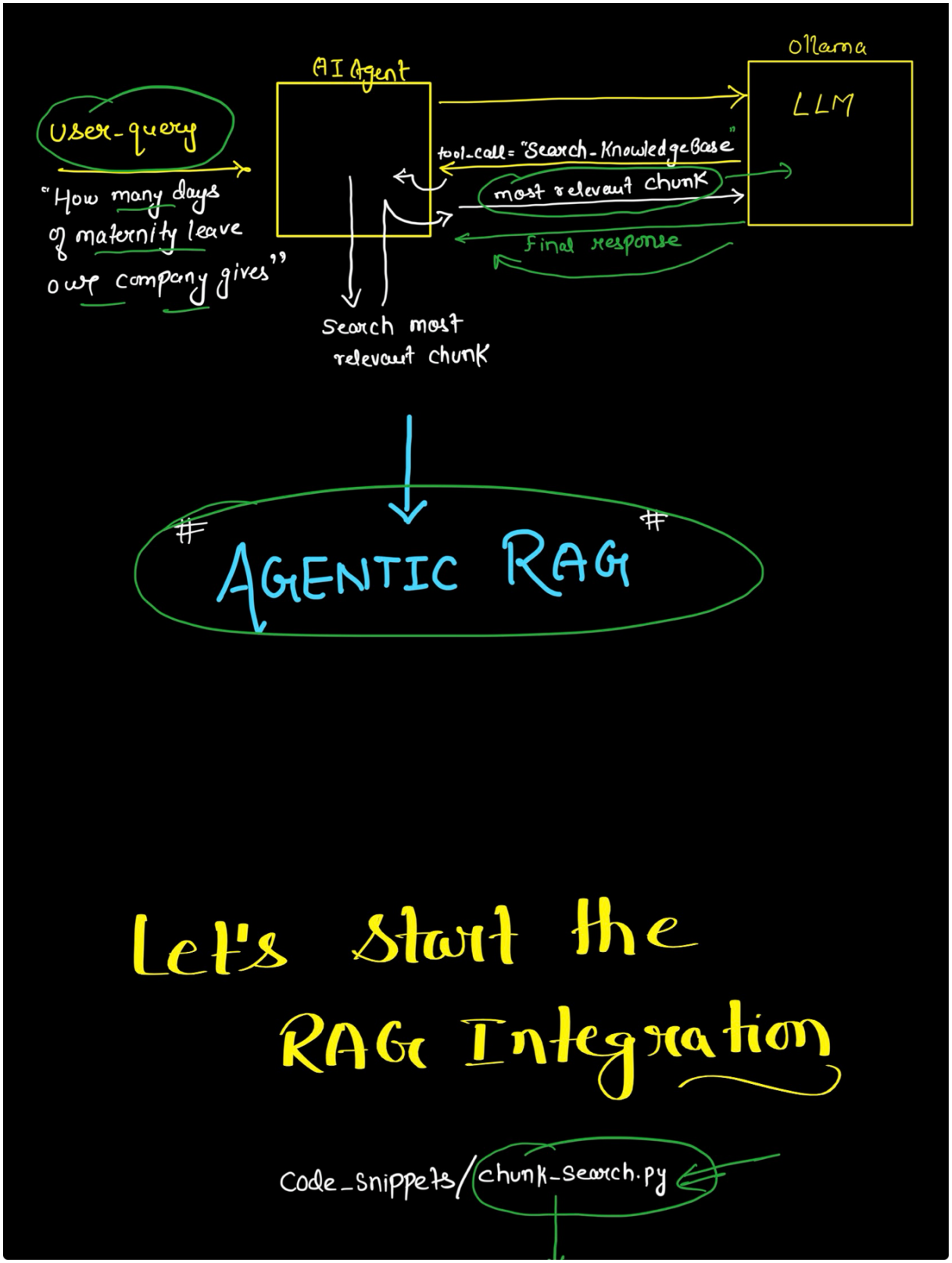
chunk_search.py

chunk_search.py

(Let's again look into this file once)

It's time to integrate the RAG in our AI-Agent

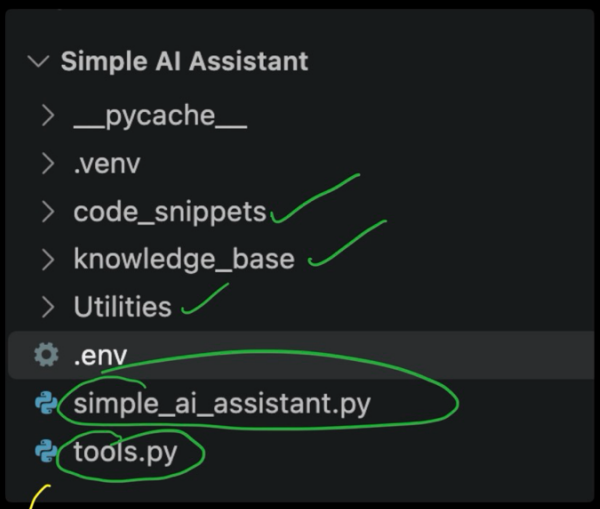
What is the industry standard of using/integrating RAG in an AI agent ???


AI Agent
Ollama
LLM
User-query
"How many days of maternity leave our company gives"
tool_call = "Search_KnowledgeBase"
most relevant chunk
Final response
Search most relevant chunk
AGENTIC RAG
Let's Start the RAG Integration
Code_Snippets/chunk_search.py

RAG Integration

Code_Snippets/chunk_search.py

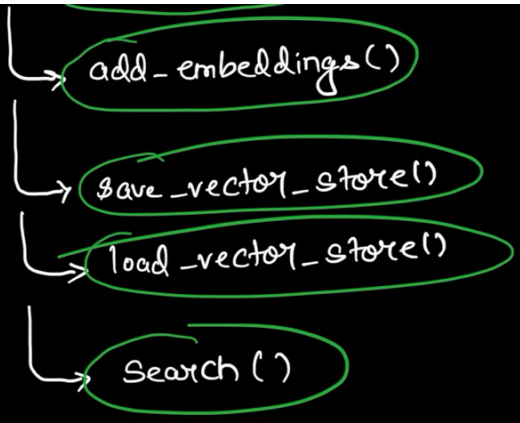
Use same code



Add → rag.py

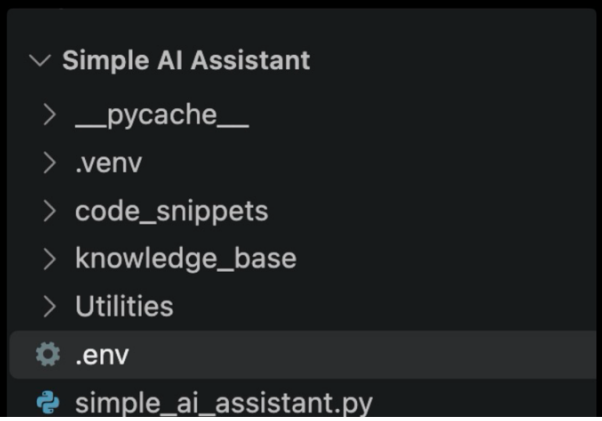
- get_embedding()
- cosine_similarity()
- load_documents()
- chunk_document()
- create_chunks()
- add_embeddings()

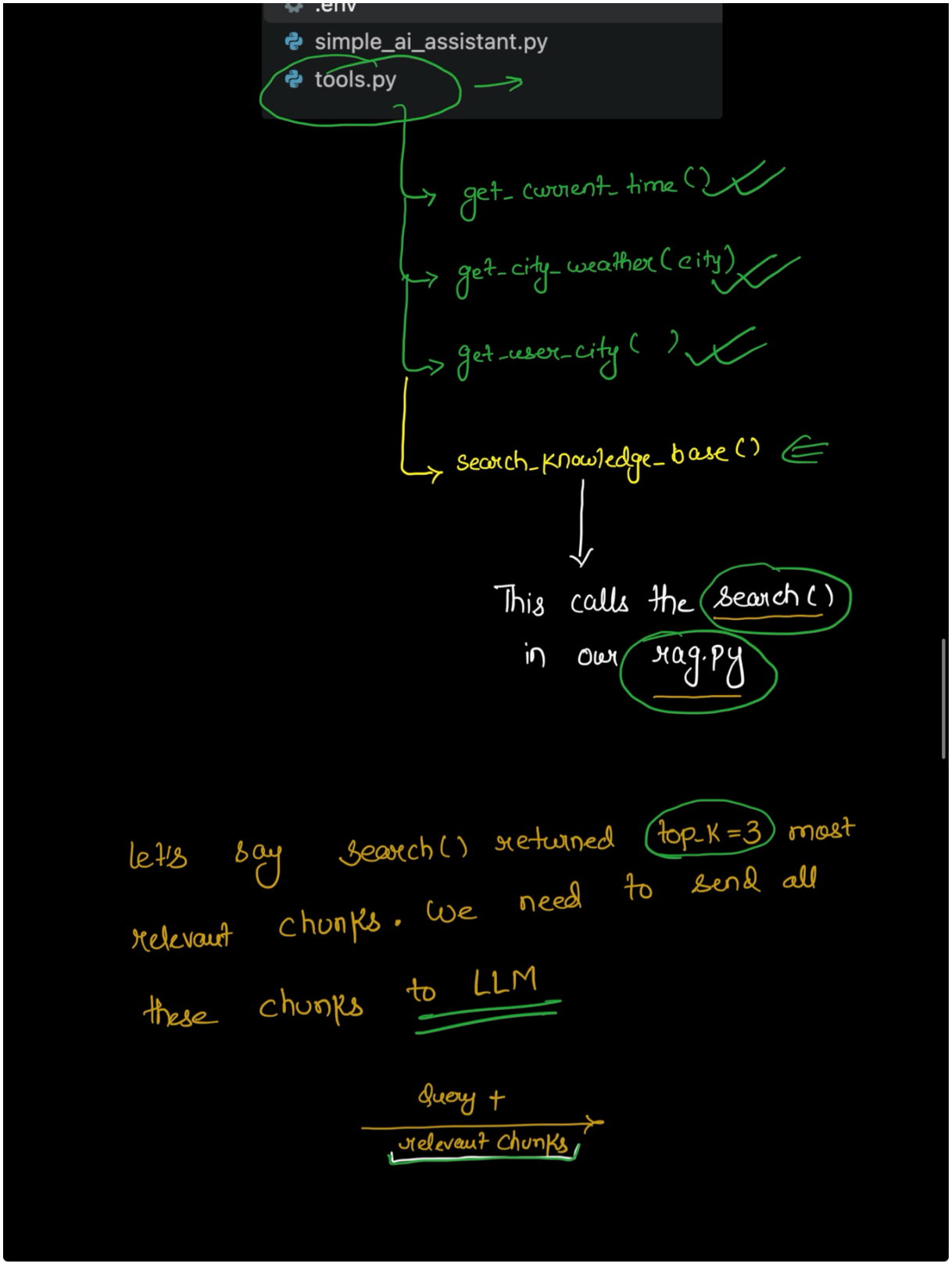
chunk_search.py



- When user_query asks something about current organisation, LLM will make a request for a tool_call = `search_knowledge_base()`

So, let's first add the tool in our tools.py




.env
simple_ai_assistant.py
tools.py
get_current_time()
get_city_weather(city)
get_user_city()
search_knowledge_base()
This calls the search()
in our rag.py
lets say search() returned top-K=3 most relevant chunks. We need to send all these chunks to LLM
query +
relevant chunks

Example:- We will send all chunks in one string to LLM

```
[leave_policy.txt #3]
Leave Policy - Maternity Leave
Maternity leave is 26 weeks of paid leave for the first two children. It can start
up to 8 weeks before the expected delivery date.

[medical_policy.txt #3]
Medical Policy - Maternity Cover
Hospitalization for childbirth is covered up to 75,000 rupees. The maternity cover
starts after 9 months of service.

[leave_policy.txt #2]
Leave Policy - Sick Leave
Employees get 10 days of paid sick leave per year. A medical certificate is required
if sick leave exceeds 2 consecutive days.
```

1. Let's write rag.py → Done
2. search_knowledge_base() → Done
   (tools.py)

```
def search_knowledge_base(query)
    results = search(query, 1)

debug mode-
    log(f" Knowledge Base Search: {query}")
    for r in results:
        log(f" {r['score']:.2f} - {r['source']}
            #{r['chunk_id']}")
```

```
blocks = [ ]

for r in results:
    sourceFileName = r['source']
    chunkId        = r['chunk_id']
    chunkText      = r['text']

    header = f"[{sourceFileName} #{chunkId}]"
    block  = f"{header}\n{chunkText}"

    blocks.append(block)

return "\n\n".join(blocks)
```

Let's also improve our system_prompt to LLM...

```
messages.append(
    {
        "role" : "system",
        "content": (
                    "You are a helpful, humble assistant. "
                    "Answer general questions (programming, science, advice) from your own knowledge. "
                    "Use a tool ONLY when the user asks for the current time, the weather, or their city. "
                    "Keep answers under 100 words."
                )
    }
)
```

Remember: I am designing our AI agent for the company where I work ex: Zorwex Technologies ~~X~~

```
"You are a helpful, humble assistant for Zorvex Technologies employees. "
"Answer general questions (programming, science, advice) from your own knowledge. "
"Use get_current_time, get_city_weather and get_user_city only for the current time, "
"the weather, or the user's city. "
```

```
"the weather, or the user's city. "
"For ANY question about Zorvex policies, benefits, leave, medical cover, HR, hiring, "
"tech processes, incident reports or travel expenses, "
"you MUST first call search_knowledge_base. "
"Answer company questions ONLY from the text that search_knowledge_base returns. "
"Do not use outside knowledge for company facts. "
"After the answer, cite the source like (leave_policy.txt #3). "
"If the returned text does not contain the answer, say exactly: "
"I could not find this in the knowledge base. "
"Keep answers under 100 words."
```

Limitations & fix

1. Small model (qwen3:4b) may skip the tool call.
2. Wrong chunk may be picked.
3. vector_store.json can become stale - if our KB changed.
4. If we change our EMBED_MODEL, vector_store.json is no longer valid.
5. We compare query embedding with every chunk which is slow if we have millions of chunks.